Database Schema Reference v13 (Full)

PH LPOST · DOCUMENT TRACKING SYSTEM # Database Schema Reference v13 (Full, Consolidated)

Scope: Departments, Users, Documents, Transmittals, Routing Transactions, Audit Logs, Undo Log, Drafts, Archive Status: Working draft — consolidates all decisions through v12; retention/purge removed per direction that backups are handled manually via NAS expansion, not automated purging

---

Design summary

A few structural decisions run through this whole schema — worth reading before the table-by-table reference:

1. **`TRANSMITTALS` is a pure, immutable log.** One row per routing event (Created, Forwarded, Received, Returned, etc.), never updated or deleted. Powers the document's routing-history timeline.
2. **`ROUTING_TRANSACTIONS` is the live, mutable parent of a single dispatch.** Each forward/initial-routing/return is one transaction. `TRANSACTION_RECIPIENTS` and `ACTIONS_NEEDED` hang off it, not off `DOCUMENTS` directly — so each dispatch has its own independent set of recipients and required actions.
3. **Single recipient vs. multiple recipients are fundamentally different processes.** A single-recipient transaction is a chain — it can be returned for revision, resubmitted, and forwarded onward to yet another department, repeatedly. A multi-recipient transaction is one-way broadcast — recipients receive and act, full stop; no return path, no forwarding onward.
4. **`AUDIT_LOGS` is a security/access trail**, not a change log — logins, views, downloads, uploads, edits, archiving. Routing changes live in `TRANSMITTALS` instead.
5. **`UNDO_LOG` is separate from both** — it logs at the operation level (one row per user action, even if that action touched five underlying rows) and is short-lived, only needed for the 1–2 minute undo window.
6. **`ARCHIVE_FOLDERS` is UI-only** — a human-organized, arbitrarily-nameable folder a document can be filed into. It has no bearing on where the file physically lives.
7. **`DOCUMENT_FILES.file_path` is a date-based real storage path** (`documents/{YYYY}/{MM}/{DD}/...`), fully independent of `ARCHIVE_FOLDERS`. It's there so the team can find and back up old files by date range on the actual disk/NAS, without needing a database query to do it.

Entities

DEPARTMENTS

FIELD	TYPE	DESCRIPTION

id	uuid (PK)	Unique identifier
name	string	Full department name
code	string	Short code, e.g. `FIN`, `LEGAL`

USERS

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
department_id	uuid (FK → DEPARTMENTS)	Which department this user belongs to
username	string	Login name
password_hash	string	Hashed password, never plaintext
role	string	staff or admin — drives which sidebar sections are visible
last_login_at	timestamp	Session/security tracking

DOCUMENTS

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier; also the legal case number
reference_no	string	Generated from date + id at creation, e.g. `20260713-<uuid>`
subject	string	The document's title
remarks	text	Free-text notes entered once at upload
origin_department_id	uuid (FK → DEPARTMENTS)	Department that originated the document
created_by_user_id	uuid (FK → USERS)	The specific user who created it
document_type	string	e.g. Budget Report, Memo, Policy
status	string	Incoming / Ongoing / For Revise / Forwarded / Released / Completed — derived, see below
is_archived	boolean	Set automatically once status reaches Completed
archive_folder_id	uuid (FK → ARCHIVE_FOLDERS, nullable)	Which UI folder it's filed into, if archived
date_created / time_created	date / time	Recorded at submission
date_released	timestamp	Set when status becomes Released
updated_at	timestamp	Auto-updated on any change

DOCUMENT_FILES

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
document_id	uuid (FK → DOCUMENTS)	Owning document
file_name	string	Original filename
file_path	string	`documents/{YYYY}/{MM}/{DD}/{file_id}_` — real storage path, auto-generated fr `uploaded_at`, for easy date-range backu NAS
file_size	int	Bytes; 50MB app-layer cap
uploaded_at	timestamp	When the file was uploaded — drives t path

TRANSMITTALS — immutable log

Append-only. Never updated after insert.

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
document_id	uuid (FK → DOCUMENTS)	The document this event happened on
transaction_id	uuid (FK → ROUTING_TRANSACTIONS, nullable)	The transaction this event was part of, if any
action	string	Created, Forwarded, Received, Returned, Resubmitted, Action Completed, Released, Completed
from_department_id	uuid (FK → DEPARTMENTS, nullable)	Department that triggered the event
remarks	text	Optional note on this event
created_at	timestamp	When it happened

ROUTING_TRANSACTIONS — mutable live state

One row per active dispatch — the parent of that dispatch’s recipients and required actions.

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
document_id	uuid (FK → DOCUMENTS)	The document being routed
	uuid (FK →	

from_department_id	DEPARTMENTS)	Sending department
type	string	initial_routing / forward / return — `return` is only ever set when the parent transaction had `recipient_count = 1`
recipient_count	int	Cached count of TRANSACTION_RECIPIENTS — drives the single-vs-multiple status logic below
status	string	open / closed
created_at	timestamp	When the transaction opened
updated_at	timestamp	Auto-updated as recipients/actions change

TRANSACTION_RECIPIENTS

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
transaction_id	uuid (FK → ROUTING_TRANSACTIONS)	Which transaction this recipient belongs to
department_id	uuid (FK → DEPARTMENTS)	Department it was sent to
status	string	pending → delivered → viewed → received
delivered_at	timestamp (nullable)	When it landed in the department's queue
viewed_at	timestamp (nullable)	When someone in that department opened it
received_at	timestamp (nullable)	When the department formally accepted it

ACTION_CODES

FIELD	TYPE	DESCRIPTION
code	string (PK)	e.g. FYAA
label	string	Full meaning, e.g. "For Your Appropriate Action"
default_text	string	Text auto-filled when this code is selected

ACTIONS_NEEDED

FIELD	TYPE	DESCRIPTI
id	uuid (PK)	Unique identifier
transaction_id	uuid (FK → ROUTING_TRANSACTIONS)	Which transaction t action belong

		to
label	string	Free-text description o the action
code	string (FK → ACTION_CODES)	Which action code applies
assigned_department_id	uuid (FK → DEPARTMENTS)	Which department i responsible
due_date	date	Optional deadline
status	string	pending / do

COMMENTS

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
document_id	uuid (FK → DOCUMENTS)	The document being commented on
user_id	uuid (FK → USERS)	Who wrote it
message	text	The comment itself
created_at	timestamp	When it was posted

NOTIFICATIONS

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
user_id	uuid (FK → USERS)	Who the notification is for
document_id	uuid (FK → DOCUMENTS, nullable)	The document it’s about
entity_type	string	e.g. “TRANSMITTALS”, “ACTIONS_NEEDED”, “COMMENTS” — what triggered it
entity_id	uuid (nullable)	The specific row, if applicable
type	string	comment / forwarded / archived / completed / action_assigned / etc.
title / message	string / text	Notification content
is_read	boolean	Read state
created_at	timestamp	When generated

AUDIT_LOGS — security/access trail

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
actor_user_id	uuid (FK → USERS)	Who did it
action	string	login, logout, viewed, downloaded, uploaded, edited, archived

entity_type	string	e.g. “DOCUMENTS”, “DOCUMENT_FILES”, “USERS”
entity_id	uuid (nullable)	The row affected — nullable for login/logout
changes	text (JSON, nullable)	Before/after values — only populated for `edited`
ip_address	string (nullable)	Where the action came from
is_undone	boolean	Only meaningful for `edited` actions
undoable_until	timestamp (nullable)	Undo window — null for non-undoable actions
created_at	timestamp	When it occurred

UNDO_LOG — operation-level undo

One row per user-facing operation, not per row changed — a single “Forward” click that touches several tables is still one `UNDO_LOG` row.

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
actor_user_id	uuid (FK → USERS)	Who performed the operation
document_id	uuid (FK → DOCUMENTS)	Which document it affected
operation_type	string	forward, return, mark_action_done, edit_document, add_comment, archive
snapshot	jsonb	`{ "created_rows": [...], "updated_rows": [{ "before": {...}, "after": {...} }] }` — everything needed to reverse it
created_at	timestamp	When the operation happened
undoable_until	timestamp	`created_at` + undo window (1–2 min)
is_undone	boolean	Prevents double-undo
undone_at	timestamp (nullable)	When it was undone, if it was

Undoing an operation deletes everything in `created_rows`, restores everything in `updated_rows` to its `before` state (one DB transaction, all-or-nothing), and appends a new `TRANSMITTALS` row (e.g. “Forward Undone”) rather than deleting the original entry — the log stays honest about what actually happened.

ARCHIVE_FOLDERS — UI-only

Purely for human organization; has no relationship to where a file physically lives on disk.

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
name	string	Folder name
parent_folder_id	uuid (FK → ARCHIVE_FOLDERS, nullable)	Enables nested subfolders
created_at	timestamp	When the folder was created

DRAFTS

FIELD	TYPE	DESCRIPTION
id	uuid (PK)	Unique identifier
document_id	uuid (FK → DOCUMENTS, nullable)	Null if a brand-new draft; set if revising an existing document
user_id	uuid (FK → USERS)	Who's editing
form_data	jsonb	Snapshot of the current form state
last_saved_at	timestamp	Last autosave time

Status derivation

`DOCUMENTS.status` is recomputed whenever a `ROUTING_TRANSACTIONS`, `TRANSACTION_RECIPIENTS`, or `ACTIONS_NEEDED` row changes. Behavior branches entirely on `ROUTING_TRANSACTIONS.recipient_count`, since single- and multi-recipient are different processes:

STATUS	SINGLE RECIPIENT (recipient_count = 1)	MULTIPLE RECIPIENTS (recipient_count > 1)
Incoming	Transaction open, recipient still `pending`	Transaction open, all recipients still `pending`
Ongoing	Recipient has reached `delivered`/`viewed`/`received`, still acting	Any recipient has reached `delivered`/`viewed`/`received`, not all `received` yet
For Revise	The recipient returns it (`type = 'return'`)	**Not reachable** — receive-only, no return path
Forwarded	The recipient forwards it onward, opening a new transaction — chain continues	**Not reachable** — broadcast recipients receive and act, they don't forward onward
Released	Transaction closes (received) and it's officially issued	Transaction closes once every recipient reaches `received`
Completed	Terminal, set explicitly. `is_archived` flips to `true`	Terminal, set explicitly. `is_archived` flips to `true`

`TRANSACTION_RECIPIENTS`'s status ladder (`pending → delivered → viewed → received`) uses the same column shape for both cases — the restriction that multi-recipient transactions can't return is enforced in application logic (the app hides/rejects the "Return" action when `recipient_count > 1`), not by a schema constraint.

Everything else holds: dashboard counters via `GROUP BY status` on `DOCUMENTS`, My Docs via `status != 'Completed'`, Archived via `is_archived = true`.

Relationships

- **DEPARTMENTS → USERS** (one-to-many)
- **DEPARTMENTS → DOCUMENTS** (originates, one-to-many)
- **USERS → DOCUMENTS** (creates, one-to-many)
- **DOCUMENTS → DOCUMENT_FILES** (contains, one-to-many)
- **DOCUMENTS → TRANSMITTALS** (logs events for, one-to-many)
- **DOCUMENTS → ROUTING_TRANSACTIONS** (dispatched via, one-to-many)
- **ROUTING_TRANSACTIONS → TRANSMITTALS** (generates, one-to-many, nullable on the log side)
- **DEPARTMENTS → ROUTING_TRANSACTIONS** (sends from, one-to-many)
- **ROUTING_TRANSACTIONS ↔ DEPARTMENTS** (via TRANSACTION_RECIPIENTS, many-to-many)
- **ROUTING_TRANSACTIONS → ACTIONS_NEEDED** (requires, one-to-many)
- **ACTION_CODES → ACTIONS_NEEDED** (classifies, one-to-many)
- **DEPARTMENTS → ACTIONS_NEEDED** (assigned to, one-to-many)
- **DOCUMENTS → COMMENTS** (has, one-to-many)
- **USERS → COMMENTS** (writes, one-to-many)
- **USERS → NOTIFICATIONS** (receives, one-to-many)
- **DOCUMENTS → NOTIFICATIONS** (triggers, one-to-many, nullable)
- **USERS → AUDIT_LOGS** (performs, one-to-many)
- **USERS → UNDO_LOG** (performs, one-to-many)
- **DOCUMENTS → UNDO_LOG** (scoped to, one-to-many)
- **ARCHIVE_FOLDERS → ARCHIVE_FOLDERS** (contains, self-referencing)
- **ARCHIVE_FOLDERS → DOCUMENTS** (stores, one-to-many)
- **USERS → DRAFTS** (owns, one-to-many)
- **DOCUMENTS → DRAFTS** (has revisions, one-to-many)